Mantık Devreleri - Ödev 3

Ders 5 / Kodlama (Deney Yaprakları 1–4)

Efe Özan - 170425062

10.11.2026

Bu belgede Ders 5 Deney Yaprakları 1–4 kapsamındaki kuramsal analizler, hesaplama tabloları ve doğrulama kayıtları yer almaktadır. Laboratuvar programları ve otomatik doğrulama betikleri `kodlar/` dizinindedir: `rakam_donusturucu.py`, `gray_donusturucu.py`, `bes_bit_ve_eslik.py`, `karakter_ve_barkod.py` ve `dogrulama_1..4`

Deney 1 - BCD, Artı-3 ve Aiken Rakam Dönüştürücüsü

1. Üç kod türü (8421 BCD, Artı-3 ve Standart Aiken 2421) için kodlayıcı ve çözümleyici `rakam_donusturucu.py` içinde tasarlandı. Artı-3 her rakama 3 ekleyerek ($d+3$), Aiken ise kendinden tümleyen standart sözlükle kodlama yapar. Boş girdi, '0'–'9' dışı karakter, 4'ün katı olmayan bit uzunluğu ve sözlük dışı 4-bitlik sözcükler gerekçeli hata fırlatarak (`ValueError`) reddedilir.

2. Zorunlu dönüşüm testleri ve 263 sayısı analizi:

- **48 ve 003 dönüşümleri:** `48` dizisi BCD ile `0100 1000`, Artı-3 ile `0111 1011`, Aiken ile `0100 1110` olarak kodlandı. `003` dizisi BCD ile `0000 0000 0011`, Artı-3 ile `0011 0011 0110`, Aiken ile `0000 0000 0011` olarak kodlandı ve baştaki sıfırlar aynen korundu. Çözümleyiciler her iki diziyi de kayıpsız geri çözdü.
- **263'ün Saf İkili vs BCD Karşılaştırması:** Onlu 263'ün saf ikili gösterimi $263_{10} = 256 + 4 + 2 + 1 =$ **100000111_2** (9 bit) iken BCD gösterimi her basamağa 4 bit ayrılarak **$0010\ 0110\ 0011_{BCD}$** (**12 bit**) olur (BCD %33.3 daha fazla alan harcar). Bu 12 bitlik BCD deseni saf ikili olarak okunursa $2^9 + 2^6 + 2^5 + 2^1 + 2^0 = 512 + 64 + 32 + 2 + 1 =$ **611_{10}** değerine karşılık gelir ve veri tamamen yanlış anlaşılır.

3. 16 adet 4-bit desenin taranması ve Aiken 0101 vs 1011 karşılaştırması:

- **BCD (8421):** 10 geçerli (`0000`–`1001`), 6 geçersiz (`1010`–`1111`).
- **Artı-3:** 10 geçerli (`0011`–`1100`), 6 geçersiz (`0000`–`0010` ve `1101`–`1111`).
- **Standart Aiken (2421):** 10 geçerli (`0000, 0001, 0010, 0011, 0100, 1011, 1100, 1101, 1110, 1111`), 6 geçersiz (`0101, 0110, 0111, 1000, 1001, 1010`).
- **Ağırlıklı Toplam Kanıtı Yetersizliği:** Aiken'de `0101` ve `1011` desenlerinin ikisinin de 2421 ağırlık toplamı $0 \cdot 2 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 =$ **5** ve $1 \cdot 2 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 =$ **5**'tir. Ancak standart Aiken'de kendinden tümleyen (self-complementing) kuralı gereği 5 rakamı 4'ün (`0100`) 1 tümleyeni olan `1011` olarak sabitlenmiştir. `0101` sözlük dışıdır. Dolayısıyla ağırlıklı toplam tek başına geçerlilik kanıtı olamaz.

4. 9 Tümleyeni Analizi ve BCD Karşı Örneği: 0–9 aralığındaki rakamların kod bitleri terslendiğinde (1 tümleyeni alındığında):

- **Artı-3:** d kodunun değeri $d+3$'tür. Bitleri terslemek $15 - (d+3) = 12 - d = (9-d) + 3$ verir. Bu tam olarak $9 - d$ rakamının Artı-3 kodudur (**10/10 doğrulandı**).
- **Aiken:** Simetrik sözlük tasarımıyla her $d \in \{0..4\}$ için tersi doğrudan $9 - d \in \{5..9\}$ sözcüğüne eşittir (**10/10 doğrulandı**).
- **BCD Karşı Örneği:** BCD kendinden tümleyen değildir (0/10). Örneğin $d = 2$ için BCD = `0010`'dir. Bitleri terslendiğinde `1101` (13_{10}) elde edilir; bu sözcük BCD'de geçerli bir rakam bile değildir. $9 - 2 = 7$'nin gerçek BCD kodu ise `0111`'dir.

YZ Önerisi Değerlendirmesi (Deney 1): YZ'nin verdiği Aiken standart sözlük tablosunu ders notlarıyla ve `0101`'in geçersizliğiyle kağıt üzerinde tek tek doğruladım.

Deney 2 - Gray Dönüştürücüsü ve Komşuluk Denetimi

1. `gray_donusturucu.py` içinde XOR tabanlı iki yönlü algoritma kodlandı:

- **İkili → Gray:** $g_{n-1} = b_{n-1}$; $g_i = b_{i+1} \oplus b_i$ ($i = n-2 \ldots 0$). Giriş bit uzunluğu korunur.
- **Gray → İkili:** $b_{n-1} = g_{n-1}$; $b_i = b_{i+1} \oplus g_i$ ($i = n-2 \ldots 0$). Burada bir önceki adımda hesaplanmış olan ikili bit (b_{i+1}) kullanılır; hazır kütüphane fonksiyonu kullanılmamıştır.

2. 0–15 Tablosu, Ardışık Komşuluk ve Kapanış Analizi: 0'dan 15'e kadar tüm 15 ardışık geçişte Hamming mesafesi tam 1'dir.

- **15 → 0 Kapanışı:** 15'in Gray'i `1000`, 0'ın Gray'i `0000`'dır. Aralarında yalnız MSB (2^3) farklıdır (**Hamming mesafesi = 1**). 4 bitlik Gray kodu dairesel (cyclic) kapanışa sahiptir.
- **9 → 0 Kapanışı (0–9 kesilmiş dizi):** 9'un Gray'i `1101`, 0'ın Gray'i `0000`'dır. 0., 2. ve 3. bitler olmak üzere tam **3 bit** farklıdır. Dizi kesilirse dairesel komşuluk özelliği bozulur.

3. Uzun Girdiler: `101110101` (9 bit) → Gray: **`111001111`** (9 bit) → Geri: `101110101`. `1000101` (7 bit) → Gray: **`1100111`** (7 bit) → Geri: `1000101`. Bit uzunluklarının korunduğu ve dönüşümün çift yönlü çalıştığı doğrulandı.

4. Hatalı Ters Yöntem ve Karşı Örnek: Ters yönde önceki ikili bit yerine komşu Gray bitlerini XOR'layan hatalı formül ($b_i = g_{i+1} \oplus g_i$) 16 sözcüğün 12'sinde yanlış sonuç verir.

- **Karşı Örnek (`1100`):** Onlu 8'in Gray kodu `1100`'dır. Doğru yöntem $b_3 = 1$, $b_2 = 1 \oplus 1 = 0$, $b_1 = 0 \oplus 0 = 0$, $b_0 = 0 \oplus 0 = 0$ $\Longrightarrow$ **1000** üretir. Hatalı yöntem ise 1. adımda $b_2 = 1 \oplus 1 = 0$, fakat 2. adımda $g_2 \oplus g_1 = 1 \oplus 0 =$ **1** hesaplayarak ilk ayrışmayı 2^1 basamağında yaşar ve hatalı olarak **1010** (10) üretir.
- **Rastgele İki Gray Sözcüğü:** Gray kodu yalnız ardışık sayılarda tek bit farkı garanti eder. Örneğin 0 (`0000`) ile 2 (`0011`) arasında 2 bit; 0 (`0000`) ile 10 (`1111`) arasında 4 bit fark vardır.

YZ Önerisi Değerlendirmesi (Deney 2): YZ'nin Gray→İkili algoritmasındaki önceki ikili biti kullanma kuralını `1100` örneğinde adım adım takip ederek doğruladım.

Deney 3 - Beş Bitte İki Bir ve Eşlik Denetimi

1. `bes_bit_ve_eslik.py` içinde iki denetleyici kuruldu:

- **5'te 2 Denetleyicisi:** 5 bitlik sözcükte tam olarak iki adet 1 bulunmasını denetler. 11000'ın özel sıfır ataması gerekçesi: 7-4-2-1-0 ağırlıklarıyla 0 normalde `00000` (sıfır adet 1) verir. Ancak kodun güvenlik koşulu her sözcükte tam iki adet 1 bulunmasıdır. 11000'ın ağırlık toplamı $7 + 4 = 11$'dir ve 0–9 tekil rakamlarıyla çakışmaz; invariantı korumak için 0'a özel olarak `11000` atanmıştır.
- **Eşlik Paketi Denetleyicisi:** 7 veri bitinin soluna p eklenerek 8 bitlik paket oluşturulur. Çift eşlikte paketteki toplam 1 sayısı çift, tek eşlikte tek olmalıdır.

2. 5'te 2 Kodunda Hata Taramaları (50 ve 100 Deney):

- **50 Tek Bit Hatası:** 10 geçerli sözcüğün her birinde 5 tek bit terslemesi sınandı. Her durumda 1 sayısı 1'e veya 3'e dönüştüğünden hataların **50/50'si (%100)** saptandı, 0 hata kaçtı.
- **100 İki Bit Hatası:** 10 sözcükte $\binom{5}{2} = 10$ çift bit terslendi. İki adet 1'in 0 yapıldığı veya iki adet 0'ın 1 yapıldığı $4 \times 10 =$ **40** hata saptandı. Ancak bir 1 ve bir 0'ın aynı anda terslendiği $6 \times 10 =$ **60** hata (%60) **KAÇTI**! Kaçan durumlarda 1 sayısı 2 kaldığı için paket başka bir geçerli rakama dönüştü (Örn: Rakam 0 (`11000`) 0. ve 2. bitler terslenirse `01100` yani Rakam 6 olur).

3. Eşlik Paketleri ve Hata Taramaları:

- `1000011` (3 adet 1) $\rightarrow$ Çift eşlik paketi: `11000011` ($p = 1$), Tek eşlik paketi: `01000011` ($p = 0$).
- `1000001` (2 adet 1) $\rightarrow$ Çift eşlik paketi: `01000001` ($p = 0$), Tek eşlik paketi: `11000001` ($p = 1$).
- **Çift Eşlikli `11000011` Paketinde Hata Taraması:** 8 tek bit hatasının **8/8'i saptandı**. $\binom{8}{2} = 28$ iki bit hatasının ise **28'i de KAÇTI (0/28 saptandı)**.
- **Tek/Çift Tersleme Kuralı:** Tek sayıda bit hatası pariteyi tersler (daima saptanır); çift sayıda bit hatası pariteyi korur (daima kaçar).

4. Düzeltme Sınırı, Hata Yeri ve 01000 Analizi: Geçersiz `01000` deseninde yalnız tek bir 1 vardır. Tek bit hatası varsayımıyla olası özgün geçerli sözcükler: `11000` (0), `01100` (6), `01010` (5) ve `01001` (4)'dir (4 olası aday).

- **Yalnızca p'nin Terslenmesi:** `11000011` paketinde yalnız p biti 0 yapılırsa paket `01000011` olur ve eşlik alarmı verir. Ancak iletilen 7 veri biti tamamen sağlamdır!
- **Kavramsal Ayrım:** *Hata saptama* (paketin kuralsız olduğunu bilme) tek eşlikle mümkündür; ancak *hatanın yerini bulma* (hangi bitin bozulduğunu bilme) ve *düzeltme* için Hamming mesafesi en az 3 olan gelişmiş kodlar gerekir. Parite hatası verinin kendisinin bozulduğunu tek başına kanıtlamaz.

YZ Önerisi Değerlendirmesi (Deney 3): YZ'nin önerdiği kombinasyonel hata taramasını `11000`$\rightarrow$`01100` geçişinde bit bit takip ederek doğruladım.

Deney 4 - Karakter Kodları ve Barkodun Bağımsız Doğrulanması

1. `karakter_ve_barkod.py` içinde H, E, L, P, boşluk ve 0–3 kümesini destekleyen ASCII ve EBCDIC (IBM-037) dönüştürücü kodlandı.

- `HELP 2031` metninin ASCII karşılığı `48 45 4C 50 20 32 30 33 31`, EBCDIC karşılığı ise `C8 C5 D3 D7 40 F2 F0 F3 F1` olarak kodlandı ve kayıpsız geri çözüldü.
- **H Karakterinin Bağımsız Ağırlık Denetimi:** ASCII kodu $48_{16} = 4 \times 16^1 + 8 \times 16^0 = 64 + 8 =$ **72_{10}**; ikili $01001000_2 = 2^6 + 2^3 =$ **72_{10}**. EBCDIC kodu $C8_{16} = 12 \times 16^1 + 8 \times 16^0 = 192 + 8 =$ **200_{10}**; ikili $11001000_2 = 2^7 + 2^6 + 2^3 =$ **200_{10}** olarak tam doğrulandı.

2. Çapraz Çözüm ve Üst Bit (MSB) Yöntemi Yetersizliği: `48 45 4C 50` EBCDIC ile ve `C8 C5 D3 D7` ASCII ile çözülmeye çalışıldığında geçersiz bayt hatası fırlatılarak reddedildi.

- **Üst Bit Neden Yetmez?** H harfinde tesadüfen ASCII (01001000) ile EBCDIC (11001000) arasında sadece 7. bit (MSB) farklı gibi dursa da bu genel bir kural değildir. Örneğin L harfinde ASCII `4C` (01001100_2) iken EBCDIC `D3` (11010011_2)'tür; MSB 1 yapılırsa `CC` olur ($CC \neq D3$). 0 rakamında ASCII `30` iken EBCDIC `F0`'dır ($B0 \neq F0$). ASCII ve EBCDIC farklı bölge (zone) yapılarına sahiptir; dönüşüm birebir tablo eşlemesi gerektirir.

3. Code 128 Üretimi ve Bağımsız Okuyucu ile Doğrulama: `python-barcode` üreticisi ile üretilen barkodlar, bağımsız optik kütüphane `pyzbar` (libzbar) ile tarandı:

- `HELP 2031` $\rightarrow$ Okunan: `HELP 2031` (9 karakter); `HELP2031` $\rightarrow$ Okunan: `HELP2031` (8 karakter). Boşluk ayrı bir çubuk sembolü olarak optik alanda taşınır.
- `000123` $\rightarrow$ Okunan: `000123` (6 karakter); `123` $\rightarrow$ Okunan: `123` (3 karakter). Barkod metin dizesi kodladığından baştaki sıfırlar korunur.

4. Üç Görünüm Deney Kaydı ve Ekranda Metin Gösterme Kanıtı:

- **1. Özgün:** 350×280 px, standart quiet zone $\rightarrow$ 5 denemede 5 başarı (%100).
- **2. Küçültülmüş:** 87×70 px (1/4 ölçek) $\rightarrow$ 5 denemede 0 başarı (%0). Piksel bulanıklığı nedeniyle zbar çubukları ayıramadı.
- **3. Sessiz Alanı Daraltılmış:** Kenar sınır ihlali $\rightarrow$ Okunabilirlik kontrast ve çerçeve koşullarına bağlı hale geldi.
- **Bağımsız Okuma Kanıtı:** Üreticiler barkod görselinin altına girdi metnini standart fontla basarlar (human readable text). Bu metin doğrudan girdi dizisinin çizilmesidir; optik çubukların taranıp deşifre edilmesiyle üretilmez. Çubuk deseni bozulsa bile alttaki metin doğru gözükür. Bu yüzden bağımsız bir optik okuyucuyla taranmadıkça barkodun optik doğruluğu kanıtlanmış sayılamaz.

YZ Önerisi Değerlendirmesi (Deney 4): YZ'nin barkod okuma önerisini `pyzbar` kütüphanesini bağımsız çalıştırarak ve görseli kırparak doğruladım.